When Quantization Stops Paying: A Capacity-Gated Roofline Model for Precision Dispatch on GPUs

Robert Zhang[1]

[1]Purdue University
[1]zhan4808@purdue.edu

Abstract

Whether a low-precision GEMM kernel actually beats its high-precision counterpart is a per-shape, per-architecture dispatch decision that the field currently makes with hand-tuned heuristics or exhaustive autotuning — and we show it is governed by a **cache-capacity condition that standard roofline analysis cannot express**, because standard roofline has no capacity term. Once a GEMM's weight working set becomes last-level-cache resident, the memory-traffic savings that motivate quantization evaporate, and any in-core dequantization cost turns the quantized path into a net loss. This reconciles two results the literature currently leaves in contradiction: quantized kernels' near-ideal small-batch speedups (e.g. MARLIN's $\sim$4$\times$ at batch $\leq$32) are the far-field regime of the same model whose near field — the L2-resident regime — yields no benefit at all.

We give a **measured cache-aware roofline model (CARM)** with three microbenchmarkable hardware parameters — effective LLC capacity, capacity-gated effective bandwidth, and dequantization throughput — and demonstrate the gate constructively on H100: a W8A8 INT8-MMA kernel with no in-core dequantization beats cuBLAS FP16 by 1.4–1.5$\times$ at decode batch sizes exactly when weights exceed the *measured* effective L2 capacity of 36 MB (not the nominal 50 MB), and loses (0.70$\times$) when they are L2-served; the win/loss boundary sits at the measured capacity. The model predicts latency to 10–18% MAPE on Triton kernels and 12% on tuned CUDA (vLLM Marlin fused-MoE FP8), and its crossover prediction (334 tokens) matches measurement (263) only against a *properly tuned* bf16 baseline — an under-tuned baseline shifts the measured crossover to 601 tokens, a methodological hazard we quantify. On real dense-model shapes (Qwen3.6-27B, where plain GEMMs are 86% of serving GPU time), the same operand-aware gate reproduces per-projection win/loss flips, and matched-precision W8A8 sustains 1.8–1.9$\times$ over tuned bf16 with no crossover, while weight-only quantization dies at the dequantization ceiling.

Two boundary conditions temper the gate. First, under full-model serving contention the L2-resident regime *collapses as a step function* once the sum of hot working sets exceeds the effective capacity — so isolated-microbenchmark crossovers (ours included) are optimistic in production, and the gate's "quantization does not pay" branch is a microbenchmark and small-model regime on today's H100, but an expanding one as LLC capacities grow (A100 40 MB $\rightarrow$ H100 50 MB $\rightarrow$ B200 $\sim$126 MB). Second, a roofline is necessary but not sufficient for precision dispatch: we document five kernel-implementation mechanisms (tile-starvation floors, wave-quantization bands, dispatch splits, occupancy floors, per-tile re-dequantization staircases) that each independently flip the win/loss sign, motivating dispatch = CARM + per-kernel predicates. MLA reconstruction GEMMs, the original subject of this study, appear as one case study of the gate. All code and data are publicly available.

1 Introduction

Whether a low-precision GEMM kernel beats its high-precision counterpart is a per-shape, per-architecture dispatch decision. Modern checkpoints ship natively quantized (MXFP4 experts, FP8 dense layers) and must run across accelerator families with different cache hierarchies and dequantization throughputs, most without native FP4 tensor cores; operator layers such as FlagGems currently make the precision decision with hand-tuned heuristics or exhaustive autotuning, neither of which ports. In real serving the stakes are concentrated in exactly this decision: in a Qwen3.6-27B serving profile, plain `aten::mm` is 86.2% of GPU time — doubling GEMM speed is a 1.76$\times$ end-to-end win, while infinite speedup on *everything else* yields only 1.16$\times$.

The literature offers contradictory guidance. MARLIN [7] reports near-ideal $\sim$4$\times$ weight-only speedup at batch $\leq$32; our measurements on MLA-scale GEMMs at batch 1 show quantized kernels *losing* to cuBLAS (0.70$\times$). Both replicate. The variable neither controls is whether the weight working set fits in the last-level cache. We show the decision is governed by a **capacity condition that standard roofline analysis cannot express**, because roofline has no capacity term:

- $W < C_{\text{eff}}$: weights are already LLC-served; quantization saves traffic that was never being paid. Speedup ≈ 1, and < 1 if the kernel dequantizes in-core.
- $W \gg C_{\text{eff}}$: weights stream from DRAM; quan-

tization saves real traffic. Speedup approaches $\min(\text{byte ratio},\ \text{dequant ceiling}/T_{\text{compute}})$ — MARLIN's regime.

Here C_{eff} is the *measured* effective residency capacity — 36 MB on H100, not the nominal 50 MB — and the two published results are the near and far fields of one model.

We formalize this as a **measured cache-aware roofline (CARM)** with three microbenchmarkable hardware parameters — effective capacity, capacity-gated bandwidth, and in-core dequantization throughput — plus an explicit per-launch fixed cost, and we validate it constructively, adversarially, and across architectures.

Contributions:

- **The capacity gate, measured** (Section 3): a 77-cell sweep (weight working set 8–128 MB $\times$ 1–512 tokens $\times$ three precisions, clock-locked, CUDA-graph timed) shows the win/loss sign flipping between 32 and 40 MB — bracketing the measured $C_{\text{eff}} = 36$ MB — at every decode-scale token count. W8A8 (INT8 MMA, no in-core dequant) goes $0.70\times \rightarrow 1.2$–$1.46\times$ across the gate; W4A16 flips at the same boundary but caps at parity, exactly the model's dequant-ceiling prediction.
- **A three-parameter predictive model** (Section 9.6): capacity-gated bandwidth ceiling, in-core dequant ceiling (r_{dequant}, set to ∞ for precisions with native MMA), and per-launch fixed cost. Regime-separated accuracy: 12–21% MAPE on the tuned cuBLAS baseline everywhere, 8–39% on quantized Triton kernels in the decode regime where dispatch operates, 12% on tuned CUDA (vLLM Marlin fused-MoE FP8).
- **Operand-aware gating on real dense shapes** (Section 3): on Qwen3.6-27B projections, each precision falls off the cache cliff at *its own operand's* size — so quantization can win by keeping its operand resident after the baseline has spilled (2.4–3.0$\times$ measured warm).
- **Boundary conditions, stated plainly** (Section 4): under serving contention the L2-resident regime collapses as a *step function* once combined working sets exceed C_{eff} — all isolated-microbenchmark crossovers, ours included, are optimistic in production; and five kernel-implementation mechanisms (tile starvation, wave-quantization bands, dispatch splits, occupancy floors, per-tile re-dequantization) each independently flip the sign, so dispatch = CARM + per-kernel predicates.
- **Cross-architecture transfer** (Section 5): a portable harness measures the parameters on any CUDA-semantics backend. Run on a real A100, it measures $C_{\text{eff}} = 31.2$ MB ($0.78\times$ nominal, the same effective/nominal ratio as H100) and predicts fp16 latency zero-shot at 19% MAPE above the gate on a self-consistent graph-timed target (13–18% on the original eager-timed 80GB dataset); naive peak-ratio scaling of kernel terms still fails ($\geq$28% vs. two-point calibration), establishing that kernel terms must be measured, not extrapolated. Below the gate the form underpredicts small L2-resident kernels (44%), a stated model-form limitation.
- **A dispatch cost model** (Section 6): holding both weight formats is memory-infeasible on 80 GB (or -33% concurrency on H200); repacking amortizes only at $\geq$20 s switch periods; JIT dequant via DRAM is 2.25$\times$ worse than not quantizing. The only zero-marginal-cost policy is quantized-primary storage — the dispatch question becomes *when to pay the dequant ceiling vs. take the quantized-compute path*, which is the question natively-quantized checkpoints force anyway.
- **A methodology audit with teeth** (Sections 8, 7): eager timing floors (15.5 μs), profiler cache flushing, and above all baseline tuning — an under-tuned bf16 baseline moves a measured precision crossover from 263 to 601 tokens, manufacturing a 2.3$\times$ wider quantization win.

MLA reconstruction GEMMs — the original subject of this study — appear in Section 8 as a case study of the gate's near field, including the INT4 negative result and the causal interventions (weight rotation, warm-state counters) that isolated the two stacked mechanisms.

2 Experimental Setup

2.1 Hardware

Primary measurements use a single NVIDIA H100 80GB SXM5 (HBM3). Cross-hardware validation uses two A100 SXM4 instances: an 80GB (HBM2e) for the 2026-06 case-study and eager-timed sweep data, and a 40GB (HBM2) on which the portable harness and the graph-timed transfer target of Section 5 were run (same GA100 die, SM count, and L2; HBM bandwidth differs, $\sim$1.5 vs. $\sim$2.0 TB/s, and is measured, not assumed). Reference peaks for H100: 3.35 TB/s HBM bandwidth, 989.4 TFLOPS BF16 tensor core, 132 SMs, 50 MB L2 cache. Reference peaks for A100: 2.0 TB/s (80GB) / 1.56 TB/s (40GB) HBM bandwidth, 312 TFLOPS BF16 tensor core, 108 SMs, 40 MB L2 cache. The 2026-08 measurements (Sections 3–6) are clock-locked (`nvidia-smi -lgc`; 1755 MHz on H100, 1410 MHz on A100); the earlier case-study measurements were not, and Section 11 quantifies the resulting variance.

2.2 Software

SGLang (commit `7ef18be`), PyTorch 2.9.1+cu128, FlashInfer 0.6.6, Triton 3.5.1, CUDA 12.8, NCU 2025.1.1.

2.3 Model Configurations

Model choice rationale. Llama-3-8B is the standard FlashInfer/Triton benchmark target. DeepSeek-V3 shapes (H=128, d_{lora}=512) define the MLA microbenchmarks: the full model is 671B and requires multi-GPU serving, so we instantiate its BMM dimensions directly without loading weights. DS-V2-Lite (15.7B, same MLA architecture) is the smallest single-

Table 1: Configurations used in this work. DS-V3 shapes (H=128, d_{lora}=512) are synthetic microbenchmark dimensions only (no weights loaded). DS-V2-Lite is used for perplexity evaluation; Qwen2.5-7B for end-to-end decomposition.

Model	H_q	H_{kv}	d	Type	KV LoRA
Llama-3-8B	32	8	128	GQA 4:1	—
DS-V3 shapes	128	128	128	MLA	512
DS-V2-Lite (15.7B)	16	16	128	MLA	512
Qwen2.5-7B	28	4	128	GQA 7:1	—

GPU MLA model available, used for perplexity evaluation. Qwen2.5-7B provides the GQA end-to-end baseline.

2.4 Methodology

Timing. CUDA events with 10+ warmup iterations, median of 100+ timed iterations. Reproducibility verified across 3 independent trials (Section 11).

FLOP counting. Standard attention: $4 \cdot B \cdot \frac{S^2}{2} \cdot H \cdot d$ ($\text{QK}^\top$ matmul + AV matmul, $\times 2$ for multiply-accumulate, $\div 2$ for causal masking). Reconstruction uses $M \times K \times N \times 2$ per BMM per head.

Bandwidth formula. Decode: $\text{BW} = (\text{KV}_{\text{read}} + Q_{\text{read}} + O_{\text{write}})/t_{\text{median}}$, where KV accounts for 99.8% of bytes.

NCU profiling. Collected with `ncu --set full --profile-from-start no` (kernel replay); overhead excluded from bandwidth runs. **Cache-state caveat:** kernel replay defaults to `--cache-control all`, which flushes GPU caches before every measured launch. All replay-based counters in this paper are therefore *cold-cache* measurements; they are valid for kernel classification (DRAM- vs. SM-bound) but carry no information about steady-state cache residency. Residency claims are based exclusively on (a) warm-loop counters collected with `--cache-control none` and (b) timing-only interventions (Section 9.5).

Timing floor. Per-launch CUDA-event measurement of these microsecond-scale kernels carries a $\sim$15.5 μs launch/eventing floor on our stack (a 0.5 MB bmm times identically to a 16 MB one). Where kernel-level conclusions are required we therefore also report CUDA-graph timing (20+ launches captured per graph, median of 50 replays), which removes per-launch CPU overhead.

Table 2: NCU metrics collected and their interpretation.

Metric	Interpretation
`dram__throughput`	HBM utilization (% peak)
`sm__throughput`	SM utilization (% peak)
`sm__warps_active`	Active warp occupancy
L1/L2 sector hit/miss counts	Cache hierarchy behavior
`launch__registers_per_thread`	Register pressure
`smsp__inst_executed_pipe_tensor`	Tensor core instruction count

3 The Capacity Gate

3.1 Model statement

For a GEMM whose recurrent operand (the weights, in inference) occupies W bytes at its stored precision, CARM predicts

$$t = t_0 + \max\Big(\frac{B}{\text{BW}(W)}, \frac{F}{P}\Big), \qquad \text{BW}(W) = \begin{cases} \text{BW}_{L2} & W < C_{\text{eff}} \\ \text{BW}_{\text{HBM}} & \text{otherwise,} \end{cases}$$

where B is bytes moved, F is FLOPs, t_0 is the per-launch fixed cost, and the compute ceiling P is the tensor-core peak for precisions with native MMA support and the in-core dequantization ceiling (r_{dequant} on packed bytes, re-paid per M-tile) for weight-only kernels that dequantize in the main loop. In the split-memory form adopted here, streamed activations and outputs are priced at BW_{HBM} while only the weight term is capacity-gated; this beats the lumped form on below-gate cells (bf16 MAPE 25.3%→20.9%). All parameters are measured, none from datasheets (Section 9.6 details the fitting; Section 5 the portable measurement). On H100 SXM5: $C_{\text{eff}} = 36$ MB (nominal 50), $\text{BW}_{L2} = 6.3$ TB/s, $\text{BW}_{\text{HBM}} = 3.15$ TB/s, $r_{\text{dequant}} = 0.50$ TB/s, $t_0 = 2.8$ μs graphed.

3.2 The gate figure

Figure 1 is the paper's central measurement: a 2D sweep of weight working set $W \in \{8 \ldots 128\}$ MB $\times$ token count $T \in \{1 \ldots 512\}$ over three precisions on the batched-GEMM family every model parameter was fit on. Three features matter. **(i) The flip location is the measured capacity.** For $T \leq 32$, W8A8 loses at 0.5–0.7$\times$ below the gate and wins at 1.19–1.46$\times$ above it, with the crossing bracketed by the 32 and 40 MB cells — independent confirmation of the NCU-derived C_{eff}. **(ii) The far field obeys the ceiling term.** W4A16 crosses at the same place but saturates at $\approx$1.0$\times$: its packed-byte dequant throughput (0.50 TB/s) makes the in-core term bind before the byte-ratio benefit can be realized. The ceiling-free byte-ratio far field is reached only by kernels without main-loop dequantization — W8A8 here, and tuned CUDA Marlin (ceiling 423 TFLOPS) in the MoE validation of Section 9.6. **(iii) The gate washes out at large** T as compute becomes the binding term, which is the same model term that predicts weight-only quantization's documented collapse beyond batch $\sim$64.

Regime-separated accuracy (reported per guardrail; blended numbers hide where models fail): the tuned cuBLAS baseline is predicted to 12–21% MAPE across the entire plane; the quantized Triton kernels to 8–39% at the decode token counts where dispatch actually operates. At $T \geq 64$ the Triton kernels drift progressively off their own rooflines (up to 81% MAPE) while cuBLAS stays at $\sim$17% — a kernel-implementation limitation, confined to the regime where no dispatcher would select them, and consistent with the per-tile re-dequantization staircase of Section 4.

Figure 1: **The capacity gate.** Speedup over tuned cuBLAS bf16 vs. normalized weight working set W/C_{eff}, one line per token count T; 77 cells, clock-locked, CUDA-graph timed (10 launches/graph, median of 30 replays), per-cell mini-autotuned. At decode-scale $T \leq 32$ the sign flips between W=32 and 40 MB — bracketing the measured C_{eff}=36 MB. W8A8 (left; INT8 MMA, dynamic act-quant inside the timed region, no in-core dequant) reaches 1.2–1.46× in the far field; W4A16 (right) flips at the same boundary but caps at parity, bound by its in-core dequant ceiling. At $T \geq 64$ both lose everywhere: the compute-bound regime that produces MARLIN's reported batch-64 collapse. Data: `profiling/gate/results_capacity_gate.json`.

3.3 Real dense shapes: the gate is operand-aware

On real Qwen3.6-27B projection GEMMs (K=8192, decode M=16), each precision falls off the residency cliff at *its own operand's* size: bf16 spills near 40 MB while W8A8's int8 operand — half the bytes — remains L2-resident to twice the nominal weight size. Between those two cliffs quantization wins not by saving DRAM traffic but by *retaining residency the baseline has lost*: measured 2.4–3.0× warm on q/o_proj (63 MB bf16 / 31 MB int8), reducing to 1.68–1.75× once rotation strips residency from both. The same data shows the in-core-dequant path (W8A16) losing at every decode size. This operand-aware form is the version of the gate a dispatcher must evaluate: compare *each candidate kernel's own* working set against C_{eff}.

3.4 The served check

The chain from microbenchmark to deployment closes end-to-end: serving the real Qwen3.6-27B on vLLM (64 sequences, cutlass fp8 W8A8 path), fp8 delivers **1.45**× decode throughput over bf16 (2931 vs. 2026 generated tok/s) — inside the 1.2–1.6× band the kernel-level model projected — and 1.20× on prefill, the compute-bound regime where the model says the benefit thins. The accuracy cost is ΔPPL $+0.032$ on wikitext-2 (7.528→7.560, 49k tokens), with per-layer relative error at the Gaussian quantization floor for every W8A8 granularity, including a kurtosis-120 outlier projection. One engine-level negative, stated plainly: shaping `max_num_batched_tokens` to wave-band edges only *hurt* decode throughput (2931→2718–2818 tok/s) — mechanism B (Section 4) does not surface in a decode-dominated engine A/B and remains a prefill batch-shaping concern.

4 Boundary Conditions

Serving contention kills the near field as a step function. Running R concurrent GEMM streams over disjoint weight copies, effective weight bandwidth for the bf16 kv_proj operand (21 MB) drops from 4.26 to 2.52 TB/s the moment total hot working set crosses C_{eff} (R=1 → 2, 21→42 MB) and is flat thereafter (2.34–2.44 TB/s out to 252 MB): the L2 tier does not degrade gracefully, it disappears. Two corollaries. First, every isolated-microbenchmark precision crossover in the literature — including Figure 1 — is optimistic under production co-residency; the near field is a microbenchmark and small-model regime on today's H100. Second, the smaller quantized operand survives contention longer (W8A8's 10.5 MB copy holds 2.9 TB/s through R=3), which is the serving-side restatement of the operand-aware gate. The regime is nonetheless *expanding* in hardware: A100 40 MB → H100 50 MB → B200 ∼126 MB → B300 ∼192 MB LLC, so the fraction of GEMM working sets below the gate grows every generation.

A roofline is necessary but not sufficient for dispatch. Five kernel-implementation mechanisms, each measured in this repo, independently flip win↔lose at fixed shape and precision: (A) persistent-grid tile starvation (Marlin's 132-CTA grid idles at $N \leq 512$: fixed 16–29 μs floor; CUTLASS immune); (B) misaligned wave-quantization bands (one shape swings 0.75–2.71× vs. M; never interpolate across band edges); (C) im-

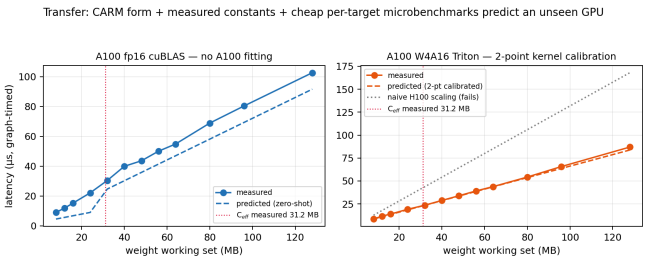

Figure 2: **Transfer to A100** (bs=1, graph-timed, A100-SXM4-40GB — the same GPU the harness constants were measured on). Left: fp16 latency predicted zero-shot from the model form plus *measured* A100 hardware constants — nothing fitted to A100 kernel measurements; the measured curve's slope break at the dotted line is C_{eff} visible in raw latency. Right: the W4A16 Triton kernel after a two-point calibration of (r_{dequant}, fixed cost) on the target; the dotted gray line scales H100 kernel terms by peak ratio and fails. Data: `profiling/portable/results_transfer_a100.json`

plementation dispatch splits (blockwise-fp8 CUTLASS path +15–21%, but $M{<}4$ falls back to a Triton kernel that loses to bf16 at decode); (D) decode occupancy floors (the register-pressure explanation was tested and refuted); (E) per-M-tile re-dequantization staircases (visible as W4A16's large-T collapse in Figure 1). Precision routing therefore composes CARM with per-kernel predicates; the model prices the physics, the predicates veto implementation pathologies.

Where the KV cache fits. KV-cache reads are outside the capacity story: decode attention shows no L2 tier at any working-set size (rotation-invariant), full attention is 2.67% of runtime in the Qwen3.6-27B profile (end-to-end ceiling $\leq$0.2%), and the fp8-KV decode kernel is bound by its own bandwidth ceiling (1.6 vs. bf16's 3.0 TB/s; 0.69–0.72$\times$ at small batch). KV precision instead enters through the *memory budget*: halving KV bytes doubles maximum concurrency at fixed memory (9$\rightarrow$18 sequences on 80 GB at the p32768d1024 operating point), which couples directly into the dispatch cost model below. Quantize KV for concurrency, not for kernel speed.

5 Cross-Architecture Transfer

The model is only useful for dispatch if its parameters can be obtained on a new backend for less than the cost of autotuning. A portable harness (`profiling/portable/measure_params.py`) measures $\{C_{\text{eff}}, \text{BW}_{L2}, \text{BW}_{\text{HBM}}, P, r_{\text{dequant}}, t_0\}$ on any backend with `torch.cuda` semantics: sweep sizes derive from the device's reported L2, bandwidths from graph-timed differential slopes, and C_{eff} from a timing-only residency-collapse detector (warm re-read bandwidth rises with buffer size as fixed cost amortizes, then breaks downward at the effective capacity) — no vendor counters required. On H100 it reproduces the NCU-derived parameters: C_{eff} 39.0 MB (counter-based band 32–40), BW_{L2} 5.26 vs. 5.33, BW_{HBM} 3.26 vs. 3.15. It also measures the *achieved* fp16 GEMM peak at 737 TFLOPS — materially below the near-datasheet 989 used earlier in this project, and the honest value for the compute-bound far field.

Applied across architectures (Figure 2), with all A100 constants *measured* by the harness on an A100-SXM4-40GB (clock-locked; $C_{\text{eff}} = 31.2$ MB $= 0.78\times$ nominal, the same effective/nominal ratio as H100; BW_{L2} 3.86, BW_{HBM} 1.51 TB/s): against a self-consistent graph-timed target re-measured on the same GPU, zero-shot fp16 lands at 19.0% MAPE above the gate and 44.0% below it — the below-gate form underpredicts small L2-resident kernels, the same residual direction as H100's 20.9% and now attributable to the model form rather than to estimated constants. Against the original eager-timed 80GB dataset the measured constants *improve* the earlier estimate-based numbers, 28.8%$\rightarrow$18.3% below / 22.8%$\rightarrow$13.0% above, despite two stated host mismatches (HBM2 vs. HBM2e bandwidth; eager launch floor). Graph timing also dissolves an artifact: the A100 kernel's apparent 58.7 μs fixed cost is 3.5 μs under CUDA graphs — it was eager overhead, not a kernel property. Kernel terms still do not transfer by scaling: peak-ratio extrapolation of r_{dequant} yields 61.4% MAPE (28.1% on the eager target) vs. 30.9% (19.9%) for a two-point calibration on the target, while the harness's directly measured $r_{\text{dequant}} = 0.394$ TB/s agrees with the value fitted from the full 2026-06 sweep (0.406) to 3%. The division of labor is the finding: *hardware terms transfer as cheap measurements; kernel terms are implementation properties that must be measured per architecture, and cannot be extrapolated.*

The gate itself transfers as capacity structure, not as a universal sign flip (`profiling/gate/results_capacity_gate_a100.json`, T$\in \{1, 16, 32\}$): on A100 the w8a8 advantage peaks exactly in the predicted asymmetric-residency band between C_{eff} and $2C_{\text{eff}}$ bf16-equivalent (2.07$\times$ at $W{=}40$ MB, T=1) and the bf16 latency curve breaks at the measured 31.2 MB, but the below-gate *loss* seen on H100 does not replicate — A100's slower bf16 baseline means dynamic-quantization overhead never dominates, so w8a8 wins even below the gate at T=1. The below-gate branch's sign is a property of baseline kernel quality, which is precisely what the model's per-kernel terms exist to capture.

6 The Dispatch Cost Model

Runtime dispatch requires the chosen format to exist in memory, and the cost of that requirement decides whether dispatch is shippable (Figure 3). **(A) Dual residency**: a second copy of a 27B model (27 GB at int8) comes out of the KV budget; at the p32768d1024 operating point (2.21 GB KV per sequence) this takes an 80 GB H100 from 9 concurrent sequences to zero, and an H200 from 36 to 24 (-33% decode throughput, since decode throughput tracks concurrency until compute-bound). **(B) Repack on switch**: measured unfused int8$\rightarrow$bf16 repack runs at 0.133 T elems/s (203 ms for 27B; fused lower bound $\text{BW}_{\text{HBM}}/3$ bytes $=$ 25 ms), so keeping overhead under 1% requires switch periods of 20 s (measured) to 2.5 s (bound) —

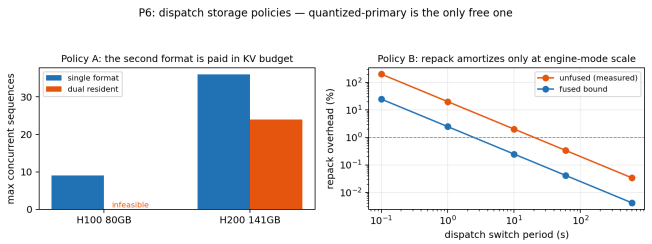

Figure 3: **Storage policies for runtime precision dispatch** (27B dense model, p32768d1024). Left: holding a second weight format is paid out of the KV budget — infeasible on 80 GB, −33% concurrency on H200 141 GB. Right: repacking on a dispatch switch amortizes below 1% overhead only at ≥20 s (measured unfused) / ≥2.5 s (fused bound) switch periods. Data: `profiling/dispatch/results_cost_model.json`.

engine-mode granularity, never per-phase or per-layer. **(C) JIT dequantization via DRAM scratch** moves $0.5{+}2{+}2 = 4.5\times$ bytes per int4 element per use vs. $2\times$ for resident bf16 — $2.25\times$ worse than not quantizing; it never pays. JIT dequantization *in-kernel* is not a fourth option: it is exactly the r_{dequant} ceiling the model already prices.

The numbers force an inversion. With bf16-primary storage, dispatch is memory-infeasible or switch-limited; the only zero-marginal-cost policy is **quantized-primary storage** with per-shape choice between (i) quantized-compute kernels (W8A8/W4A4) and (ii) dequant-in-kernel bf16 compute priced by r_{dequant}. The dispatcher's question is then not *"when do we quantize?"* but *"when do we pay the dequant ceiling vs. take the quantized-compute path?"* — which is the question natively-quantized checkpoints (MXFP4 experts, FP8 dense) pose regardless. KV precision couples in through the same budget: fp8 KV doubles feasible concurrency (Section 4) and is worth more to serving throughput than any per-kernel win in this paper.

7 Attention Kernel Validation

Before presenting our MLA findings, we validate the profiling methodology by comparing FlashInfer and Triton attention kernels on Llama-3-8B, a well-studied GQA configuration. This reproduces known performance gaps and establishes the measurement infrastructure used in subsequent sections.

7.1 Decode: Bandwidth Scaling

Both backends are firmly memory-bound (Table 3). FlashInfer peaks at 2,987 GB/s (89% of 3.35 TB/s) while Triton peaks at 2,669 GB/s (80%). The gap narrows from $2\times$ at bs=1 (launch-overhead dominated) to $1.12\times$ at bs=256 (bandwidth-saturated); Figure 4 shows the full scaling curve.

FlashInfer approaches 3,000 GB/s at $L_{kv} = 8192$, saturating HBM at 89.6% of peak (Table 4).

Table 3: Decode performance, Llama-3-8B GQA, $L_{kv} = 2048$.

BS	FI (ms)	Tri (ms)	FI BW	Tri BW	Speedup
1	0.028	0.057	298	147	**2.03**×
4	0.032	0.058	1056	582	1.82×
16	0.058	0.070	2317	1910	1.21×
64	0.187	0.218	2870	2470	1.16×
128	0.364	0.416	2957	2585	1.14×
256	0.720	0.806	**2987**	**2669**	1.12×

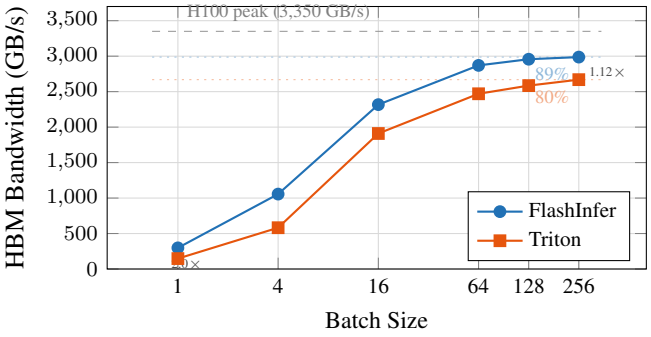

Figure 4: Decode bandwidth vs. batch size (Llama-3-8B, L_{kv}=2048). Both backends approach HBM saturation; FlashInfer peaks at 89% of H100 peak. The gap narrows from $2\times$ at bs=1 to $1.12\times$ at bs=256 as launch overhead becomes negligible relative to streaming KV reads.

7.2 Decode: NCU Analysis

The occupancy paradox. Table 5 shows FlashInfer uses 183 registers/thread ($2.4\times$ Triton's 76), yielding only 12.2% active warps vs. 35.4%. Yet FlashInfer achieves 84% DRAM throughput vs. 76%. This shows that *memory access pattern quality dominates occupancy* for bandwidth-bound kernels: FlashInfer's fused design with coalesced, pipelined accesses extracts more bandwidth per warp than Triton's higher-occupancy two-phase approach.

Two-phase overhead. Triton splits attention into Stage 1 (KV scatter-gather, 217 μs) and Stage 2 (softmax reduction, 10 μs). Stage 2 adds 4.4% overhead but achieves 73.8% occupancy with only 30 registers, a well-optimized reduction kernel.

L2 irrelevance at scale. Both backends show <1% L2 hit rate. The KV cache at bs=64 is $64 \times 2048 \times 8 \times 128 \times 2 = 256$ MB, far exceeding the 50 MB L2. GQA's 4:1 head sharing provides no cache benefit at this scale.

7.3 Prefill: TFLOPS Scaling

FlashInfer peaks at **552 TFLOPS** (55.8% of 989.4 T peak); Triton peaks at 209 TFLOPS (21.1%) (Table 6, Figure 5). The $2.6\times$ gap is consistent across all configurations and *widens* with sequence length, pointing to a structural difference rather than an incidental one.

Table 4: Decode performance, Llama-3-8B GQA, bs=64, varying L_{kv}.

L_{kv}	FI (ms)	Tri (ms)	FI BW	Tri BW	Speedup
256	0.037	0.065	1859	1053	**1.76**×
512	0.059	0.069	2273	1950	1.17×
1024	0.102	0.113	2648	2389	1.11×
2048	0.187	0.217	2875	2476	1.16×
4096	0.360	0.409	2984	2624	1.14×
8192	0.716	0.790	**3000**	**2720**	1.10×

Table 5: NCU metrics, decode, bs=64, $L_{kv} = 2048$.

Metric	FlashInfer	Triton (Stage 1)
Kernel	`BatchPrefill..`	`_fwd_grouped..`
DRAM util.	**84%**	76%
SM util.	17%	28%
Regs/thread	**183**	76
Active warps	12.2%	**35.4%**
Tensor insts	2.16M	2.10M
L2 hit rate	0.4%	1.0%
Duration	**191** μs	217 μs

7.4 Prefill: NCU Root Cause Analysis

7.4.1 Finding: TMA vs. Global Loads (Not "Cache Hit Rate")

A naive reading of NCU's L1 sector counters (Table 7) yields: FlashInfer 97% L1 hit rate, Triton 0.1%. **This is misleading.**

FlashInfer's Hopper kernel uses TMA (Tensor Memory Accelerator) [11], a dedicated hardware unit that performs asynchronous bulk tensor copies directly from HBM/L2 into shared memory, *bypassing the L1 global load data path entirely*. The 108K L1 global load sectors are residual metadata accesses (indptr arrays, scalar parameters), not QKV tensor data. The 37.8M sectors in Triton represent the *actual* QKV working set flowing through L1 via standard `ld.global` instructions.

The correct comparison is at L2: FlashInfer achieves 86.4% hit rate vs. Triton's 72.0%. FlashInfer's cooperative grid launch (132 blocks = 1 per SM) creates a controlled L2 working set; Triton's 2048-block grid generates 83% more L2 misses (9.9M vs. 5.4M sectors) due to wave-quantization thrashing.

This distinction is architecturally fundamental. TMA access is not "better caching" but a *different hardware data path* unavailable to Triton's compiler, which generates standard `ld.global` PTX.

7.4.2 Finding: No Register Spills (Occupancy-Only Bottleneck)

Triton's prefill kernel compiles to 255 registers, the `sm_90` hardware maximum. A natural hypothesis is that registers spill to local memory, adding latency. We tested this at three levels:

1. **NCU local memory counters**: The local memory wavefront counters (`l1tex..mem_local_op_{ld,st}.sum`) returned `n/a` on all Hopper kernels (known NCU limitation on sm_90).

2. **PTX inspection**: Zero `.local` directives, zero `st.local` / `ld.local` instructions in the compiled PTX.

3. **CUBIN resource dump** (`cuobjdump --dump-resource-usage`): `LOCAL:0` for `_fwd_kernel`.

Verdict: Triton's 255-register kernel does *not* spill. The compiler fits all live variables within the register file at the cost of maximal register allocation. The performance impact is purely occupancy: fewer warps per SM means fewer opportunities to hide memory latency.

Reducing Triton's register count (e.g., via different tiling or explicit `num_stages` tuning) could improve occupancy and partially close the prefill gap, but the TMA access pattern difference (Section 7.4.1) would remain.

Table 6: Prefill performance, Llama-3-8B GQA. TFLOPS uses corrected formula ($4BS^2Hd/2$).

BS	S	FI (ms)	Tri (ms)	FI TF	Speedup
1	512	0.024	0.040	88	1.63×
1	1024	0.033	0.069	264	2.12×
1	2048	0.089	0.214	387	2.41×
1	4096	0.283	0.730	485	2.58×
4	2048	0.307	0.753	448	2.45×
4	4096	0.995	2.698	**552**	**2.71**×
16	2048	1.238	2.822	444	2.28×
16	4096	4.299	10.531	512	2.45×

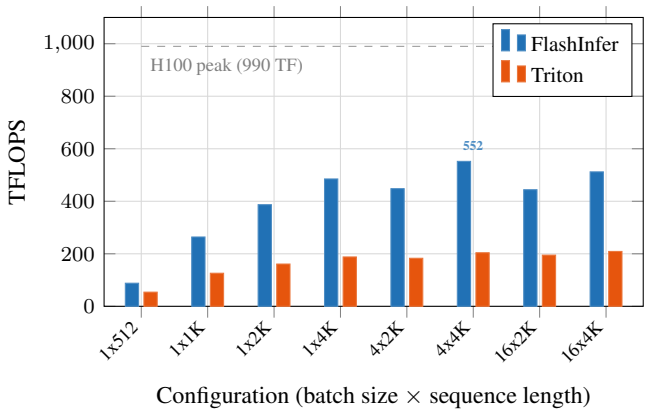

Figure 5: Prefill compute throughput (Llama-3-8B GQA). FlashInfer achieves 2.1–2.7× higher TFLOPS across all configurations, peaking at 552 TFLOPS (56% of peak). The gap widens with sequence length due to FlashInfer's TMA hardware loads and cooperative grid launch (Section 7.4).

7.4.3 Finding: Cooperative Grid vs. Wave Quantization

FlashInfer launches exactly 132 thread blocks (one per SM) using CUDA cooperative launch semantics. This eliminates wave-quantization effects and enables persistent-kernel-style

Table 7: NCU metrics, prefill, bs=4, $S = 2048$.

Metric	FlashInfer	Triton
Kernel	`PrefillWith...`	`_fwd_kernel`
SM util.	**52.0%**	32.7%
DRAM util.	13.1%	9.3%
Regs/thread	168	**255** (max)
Active warps	14.1%	12.5%
Grid size	132 (cooperative)	2048
L1 global ld sectors	**108K**	37.8M
L2 hit rate	**86.4%**	72.0%
Tensor insts	2.23M	2.23M
LOCAL mem (CUBIN)	**0 bytes**	**0 bytes**
Duration	**368** μs	909 μs

execution where each block processes multiple tiles sequentially with full shared memory and register file utilization.

Triton launches 2,048 blocks (15.5× more), dispatched in multiple waves. Each wave evicts the previous wave's L2 residency, explaining the 14-point L2 hit rate gap (86.4% vs. 72.0%).

8 Case Study: MLA Reconstruction and the INT4 Negative Result

This section and the next preserve the study that discovered the gate: the MLA reconstruction workload, its INT4 mitigation attempt, the causal interventions (weight rotation, warm-state counters, CUDA-graph timing) that isolated partial L2 residency and the in-core dequantization ceiling as the two stacked mechanisms, and the W8A8 kernel that confirmed the model constructively. Sparse/MoE operators were subsequently verified (FlagGems, 2026-07) to be outside the capacity story — they are not cache-limited — so these results stand as the gate's near-field case study rather than as claims about MLA serving in general.

Having established confidence in our profiling infrastructure, we turn to our central question: how much of MLA's attention-layer time is spent on reconstruction?

Multi-head Latent Attention [4] compresses KV cache to a low-rank latent $c_t \in \mathbb{R}^{d_{\text{lora}}}$ with $d_{\text{lora}} = 512$. During decode, full-dimensional K and V are *reconstructed* via weight-absorbed batch matrix multiplications (BMMs):

$$\text{BMM1: } Q'_h = Q_h W_h^{\text{kc}}, \quad W_h^{\text{kc}} \in \mathbb{R}^{d_{\text{nope}} \times d_{\text{lora}}}$$
$$\text{BMM2: } O_h = A_h W_h^{\text{vc}}, \quad W_h^{\text{vc}} \in \mathbb{R}^{d_{\text{lora}} \times d_v}$$

where $h = 1, \ldots, H$ heads. These are batched GEMMs with batch dimension H, independent of KV sequence length.

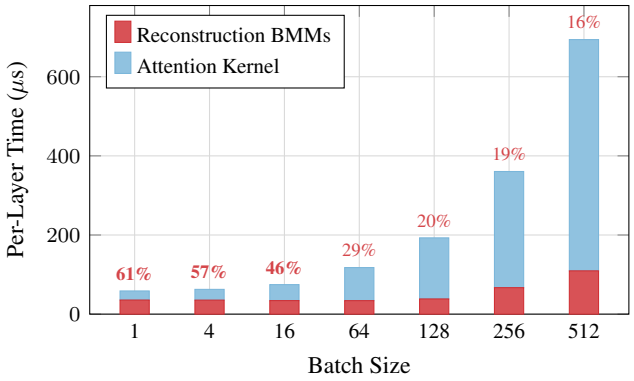

Figure 6: MLA attention-layer time decomposition per layer (DeepSeek-V3 shapes, L_{kv}=2048). Reconstruction BMMs (red) dominate at small batch sizes (**61% at bs=1**) and remain significant at bs=512. Reconstruction cost is nearly constant (∼35 μs), independent of KV length; attention scales linearly.

8.1 Cost Breakdown: DeepSeek-V3 Architecture

We profile reconstruction BMMs separately from FlashInfer MLA attention on DeepSeek-V3-scale shapes ($H = 128$, $d_{\text{nope}} = 128$, $d_{\text{lora}} = 512$, $d_v = 128$).

Table 8: MLA reconstruction overhead per layer, DeepSeek-V3 shapes, $L_{kv} = 2048$. Reconstruction cost is independent of KV length.

BS	Recon (μs)	Attn (μs)	Recon %	Model (ms)
1	**35.6**	23.0	**60.7%**	**2.17**
4	35.4	27.2	56.6%	2.16
16	34.2	40.4	45.8%	2.08
64	34.1	83.6	29.0%	2.08
128	38.4	154.4	19.9%	2.34
256	66.9	293.6	18.6%	4.08
512	109.4	584.5	15.8%	6.68

Key finding: At batch size 1, reconstruction BMMs consume **60.7%** of attention-layer time (35.6 μs vs. 23.0 μs for the attention kernel itself) (Table 8, Figure 6). Across 61 layers, this totals **2.17 ms per token**, a fixed per-query overhead invisible to standard attention benchmarks.

8.2 Roofline Analysis: Memory-Bound at All Batch Sizes

Arithmetic intensity peaks at 93 (bs=128+), well below the crossover point of 295 (Table 9, Figure 7). **All reconstruction BMMs are memory-bound.** Achieved bandwidth ranges from 952 GB/s to 2,114 GB/s (28–63% of HBM peak), leaving significant room for kernel optimization.

Critical implication: Because reconstruction is memory-bound and weight-dominated (weight matrix is $128 \times 128 \times 512 \times 2 = 16$ MB per BMM, fixed regardless of batch size), reducing weight precision directly reduces the memory bandwidth bottleneck.

Table 9: Arithmetic intensity and achieved bandwidth for reconstruction BMMs. H100 roofline crossover: AI = 295 (990 TFLOPS / 3.35 TB/s).

BS	AI (BMM1)	BW_1	AI (BMM2)	BW_2
1	**0.97**	954	**0.97**	952
16	15.5	1133	15.5	1137
64	62.1	1592	62.1	1607
128	**93.0**	**1837**	**93.0**	**2114**
256	93.0	1741	93.0	1770

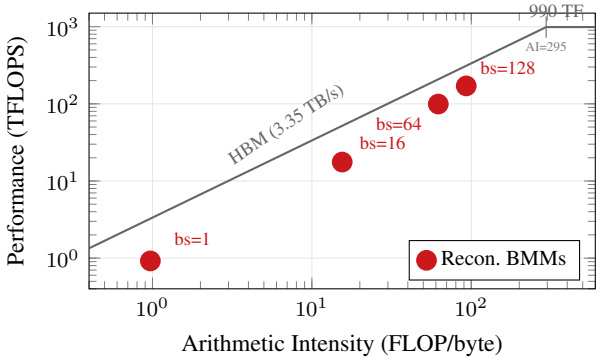

Figure 7: H100 roofline with MLA reconstruction BMMs. All operating points fall on the memory-bound slope, well below the compute ceiling (crossover at AI=295). Even at bs=128 (AI=93), reconstruction achieves only 55% of the memory ceiling.

9 Case Study (cont.): INT4 Mitigation and the L2 Cache Barrier

Reconstruction BMMs are memory-bound at all practical batch sizes, and their weight matrices dominate the data transfer. Reducing weight precision should therefore yield proportional speedups. We evaluate INT4 weight-only quantization of W^{kc} and W^{vc}, addressing two questions: does it preserve quality, and does it deliver the expected performance gain?

9.1 Perplexity Evaluation

We evaluate on DeepSeek-V2-Lite (15.7B parameters) using wikitext-2 test set (308K tokens, stride-512 sliding window, max length 2048). Three configurations:

Table 10: Wikitext-2 perplexity under INT4 quantization. `kv_b_proj` is the reconstruction weight (W^{kc}/W^{vc} combined).

Configuration	PPL	Δ	Weights quantized
FP16 baseline	**5.727**	—	0
INT4 selective	5.777	**+0.051**	27 (kv_b_proj)
INT4 all linear	**11.784**	+6.057	5,209

Selective INT4 quantization of reconstruction weights adds only +0.051 perplexity (Table 10, Figure 8), an order of magnitude below the typical 0.5 quality threshold. Naive INT4 of all linear weights, by contrast, more than doubles perplexity.

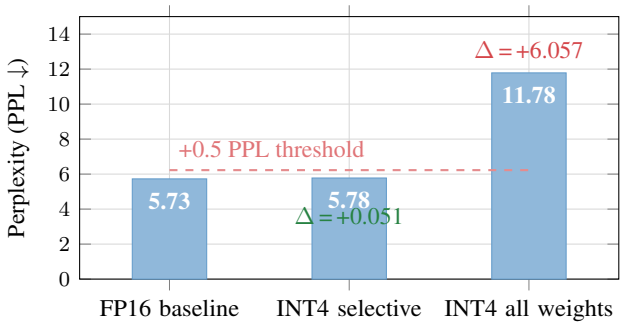

Figure 8: Wikitext-2 perplexity under INT4 quantization (DeepSeek-V2-Lite). Selective INT4 of reconstruction weights (`kv_b_proj`) adds only +0.051 PPL, well within the +0.5 quality threshold. Naive INT4 of all linear weights severely degrades quality.

9.2 Why Reconstruction Weights Are Quantization-Friendly

The W^{kc} and W^{vc} matrices are *projection weights* that map between a compressed 512-dimensional latent space and 128-dimensional head spaces. Several factors explain their quantization robustness:

1. **Low-rank structure**: The latent space was trained to be compressible; the projection matrices inherit smooth spectral properties.
2. **Redundancy**: With $H = 128$ heads each projecting from the same 512-dim latent, per-head reconstruction errors are averaged across heads before affecting output.
3. **Post-softmax attenuation (hypothesized)**: BMM2 operates on attention-weighted values; quantization noise is attenuated by the softmax distribution's sparsity.

9.3 Theoretical Speedup

For memory-bound operations, speedup from weight quantization equals the ratio of total data transferred:

$$\text{Speedup} = \frac{\text{Weight}_{\text{FP16}} + \text{Activation}_{\text{FP16}}}{\text{Weight}_{\text{INT4}} + \text{Scale/ZP} + \text{Activation}_{\text{FP16}}} \quad (1)$$

At batch size 1 (latency-critical single-query serving), the roofline predicts **3.94× speedup** (Table 11), which would reduce the 2.17 ms full-model reconstruction overhead to ∼0.55 ms.

9.4 Measured Kernel Performance

We implemented a custom batched W4A16 Triton kernel that fuses the head dimension into the grid, avoiding 128 separate

Table 11: Theoretical INT4 speedup for reconstruction BMMs. Weight bytes dominate at small batch sizes.

BS	Wt (MB)	Act (MB)	INT4 eq. (MB)	Speedup
1	16.00	0.03	4.00	**3.94**×
4	16.00	0.13	4.00	3.89×
16	16.00	0.50	4.00	3.67×
64	16.00	2.00	4.00	3.00×
128	16.00	4.00	4.00	2.50×
256	16.00	8.00	4.00	2.00×

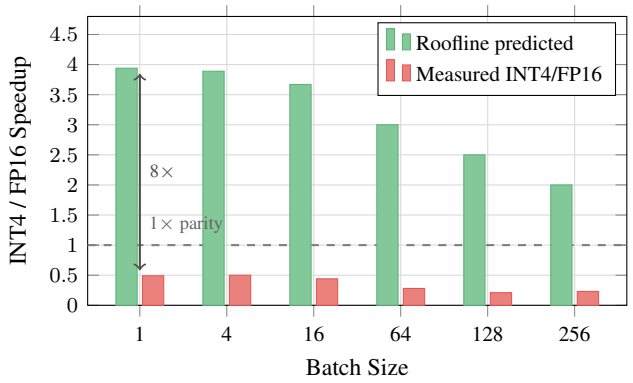

Figure 9: The L2 cache barrier. Roofline analysis predicts 2–3.9× INT4 speedup assuming HBM-bound operation. Measured performance shows 0.2–0.5×: the INT4 kernel is *slower* than cuBLAS FP16. The 16 MB reconstruction weight fits within the measured ∼36 MB effective L2 capacity; in steady state ∼75% is served from L2 at an effective 3.5–4.6 TB/s, removing most of the HBM savings that motivate quantization (and the dequant-bound kernel forfeits the rest).

kernel launches. The kernel dequantizes INT4 weights to FP16 in registers and uses tensor core `tl.dot` for the matmul.

Table 12: Measured INT4 kernel performance vs. cuBLAS FP16 `torch.bmm`. "FP16 loop" = 128 individual `torch.mm` calls.

BS	FP16 bmm (ms)	INT4 Tri (ms)	FP16 loop (ms)	Ratio INT4/FP16
1	**0.036**	0.073	2.194	**0.49**×
4	0.037	0.073	2.203	0.50×
16	0.036	0.082	2.187	0.44×
64	0.036	0.129	2.215	0.28×
128	0.040	0.187	2.211	**0.21**×
256	0.070	0.302	2.223	0.23×

The INT4 kernel is 2× slower than cuBLAS FP16, not 3.9× faster (Table 12, Figure 9). It is, however, 30× faster than a per-head FP16 loop (0.073 ms vs. 2.19 ms), confirming that the batched grid approach itself is effective. The bottleneck is competing with cuBLAS, not the batching strategy.

9.5 The L2 Cache Barrier: Why Roofline Fails for Small-Matrix MLA Reconstruction

The 8× gap between roofline-predicted (3.9×) and measured (0.49×) performance has three contributing factors. No single factor is sufficient: a fixed-shape rotation intervention (below) shows that removing factor (1) alone still leaves INT4 at or below FP16 parity.

(1) Partial L2 residency weakens the HBM-bound assumption. The standard roofline model [14] assumes data is served from HBM at 3.35 TB/s. But the total reconstruction weight per BMM is $128 \times 128 \times 512 \times 2 = 16$ MB, within H100's 50 MB L2 cache. Warm-loop counters (`ncu --cache-control none`, steady-state launch after 30 warm-ups) show `torch.bmm` re-reads only 4.0 MB of the 16.8 MB working set from DRAM per launch — ∼75% L2-served — at an effective serving bandwidth of 3.5–4.6 TB/s. (The often-quoted ∼12 TB/s aggregate L2 bandwidth is not achieved by these kernels; CUDA-graph timing puts the incremental L2-era slope at 6.3 TB/s and total-bytes throughput at 3.5–4.6 TB/s.) Reducing weight precision from FP16 to INT4 saves HBM bandwidth that *was largely not the bottleneck*.

We validate the size dependence by scaling the weight matrix across the L2 boundary. Holding H=128 and d_{nope}=128 fixed, we sweep d_{lora} from 256 to 4096, producing FP16 weight sizes from 8 MB to 128 MB. The event-timed INT4/FP16 time ratio drops from 1.91× at 8 MB to 1.08× at 128 MB on the main stack; Figure 13 (left) shows the same knee on the audit stack. Warm-state DRAM-read counters locate the actual residency cliff at **32–40 MB** (re-reads collapse to 2.2 MB at 32 MB weights, then snap to ∼100% of weight size at 40 MB and above; Figure 12, right): the *effective* LRU residency capacity is ∼36 MB, below the nominal 50 MB, and the knee should be read against that corrected boundary. Two confounds inflate the event-timed curve's flatness below 32 MB: a ∼15.5 μs per-launch timing floor (Section 2.4; Figure 11, left), and cuBLAS switching `nvjet` tile variants across the sweep. CUDA-graph timing removes the floor and shows FP16 latency scaling smoothly at 6.3 TB/s below the cliff and 2.7 TB/s above it, with kernel-level INT4/FP16 ratios of 1.9× (16 MB) → 1.0–1.1× (≥40 MB): **INT4 reaches parity above the cliff but never wins**. Two independent counters corroborate the tier change: below the cliff FP16's logical-byte throughput exceeds the 3.35 TB/s HBM peak — only possible if L2-served — at an effective 3.5–4.4 TB/s, settling to ∼2.5–2.7 TB/s above it (Figure 12, left); and useful FLOP throughput peaks at ∼4.6 TFLOPS, 0.5% of the 989 TF peak, so neither kernel is compute-throughput-bound and INT4's high SM utilization is dequantization overhead, not useful work (Figure 11, right).

Weight-rotation intervention (residency removed at fixed shape). To test whether L2 residency is *sufficient* to explain INT4's failure, we hold the MLA shape fixed (16 MB weights) and cycle each launch through k independent weight copies inside a CUDA graph (Figure 13, right). With k=1–2 (working set ≤32 MB, L2-resident) FP16 takes 4.8 μs per BMM; with k=6 (96 MB working set, residency destroyed) FP16 rises

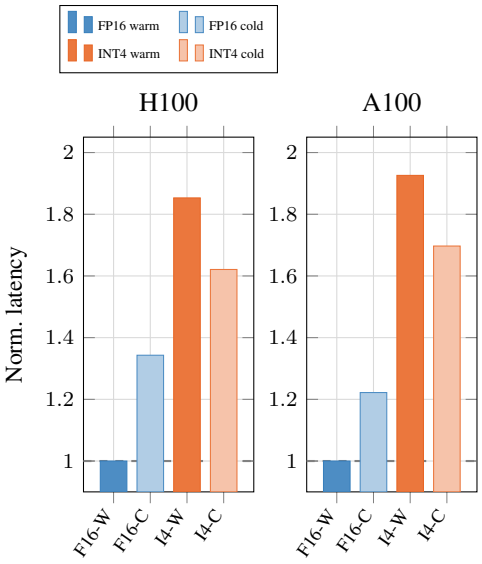

Figure 10: Cache-residency intervention at fixed MLA reconstruction shape (BS=1, normalized to FP16-warm). H100: eviction adds +4.8 μs to FP16 (= full 16 MB weight reload at HBM rate) and +3.9 μs to INT4, of which $\sim$2.6 μs is common-mode eviction overhead (expected INT4 reload: 1.3 μs). The FP16 delta supports L2-residency dependence of the FP16 path; note that absolute deltas, not relative percentages, are the comparable quantity here since the two baselines differ $\sim$2.6$\times$.

to 8.3 μs — the +3.5 μs delta matching an HBM-rate weight reload. INT4 is cache-state-insensitive (9.1 vs. 9.3 μs). Critically, **INT4 remains 1.12$\times$ slower than FP16 even with FP16 forced to HBM**. L2 residency accounts for the gap growing from 1.12$\times$ to 1.91$\times$, but eliminating it would not make this INT4 kernel win: the kernel runs 2.7$\times$ above its memory entitlement ($\sim$3.4 μs for 4.2 MB at HBM rate) due to factor (2).

Controlled cache-residency intervention. To establish L2 residency as a causal factor rather than a correlate of matrix size, we hold the reconstruction shape fixed (H=128, d_{nope}=128, d_{lora}=512) and vary only the cache state immediately before each timed launch. Under the warm condition, 20 untimed kernel executions load the 16 MB weight tensor into L2 before measurement. Under the cold condition, a memory sweep (4$\times$ L2 capacity: 256 MB on H100, 192 MB on A100) evicts L2 contents immediately before the timed launch, with explicit synchronization between eviction and measurement.

Figure 10 shows this intervention on both H100 and A100. We report *absolute* latency deltas, since the FP16 and INT4 baselines differ by $\sim$2.6$\times$ and relative percentages are therefore not comparable across kernels. On H100, forced eviction adds +4.8 μs to FP16 — matching a full 16 MB weight reload at HBM rate (16 MB / 3.35 TB/s = 4.8 μs) — and +3.9 μs to INT4. INT4's expected reload is only $\sim$1.3 μs (4.2 MB packed), so roughly +2.6 μs of its delta is common-mode overhead from the eviction procedure itself (DRAM write-back of the dirty eviction buffer); a control with an 8 MB eviction buffer (too small to displace the weights) adds $<$0.3 μs to FP16, confirming the eviction buffer, not measurement noise, produces the common-mode term. The FP16-specific component (+4.8 μs $\approx$ exact HBM reload time) supports L2-residency dependence of the FP16 path; an earlier version of this analysis reported the same data as relative percentages (FP16 +34% vs. INT4 +17%), which overstated the asymmetry. NCU counter attribution per tensor allocation is not available in current tooling; the warm-loop `--cache-control none` DRAM-read counters and the rotation intervention above are the primary causal evidence, with this eviction experiment as corroboration.

Replay-based NCU profiling across the same sweep classifies the two kernels — with the caveat that these counters are *cold-cache* (cache-flushed replay, Section 2.4) and therefore speak to kernel character, not to residency. On H100, the FP16 cuBLAS kernel (`nvjet`, TMA-based) is memory-side at every size: SM utilization stays below 15%, and DRAM utilization rises smoothly from 35% at 8 MB to 83% at 128 MB — a duration-amortization effect (short kernels cannot sustain peak DRAM), *not* an L2-boundary knee; the steepest rise occurs below 32 MB. The INT4 Triton kernel is the opposite at every size: SM utilization scales from 33% to 79% (dequantization dominates) while DRAM utilization stays below 23%. On A100 the directions are the same (FP16 DRAM 43%$\rightarrow$66%, INT4 SM 51%$\rightarrow$75%). There is thus no within-kernel memory-to-compute *transition* anywhere in the sweep: FP16 is always memory-side and INT4 always dequant-compute-side; what changes with size is FP16's serving *tier* (L2 below $\sim$36 MB effective capacity, HBM above), which is visible only in the warm-state counters and timing interventions above. INT4 reads exactly 4$\times$ fewer DRAM bytes as expected, but converting that savings into latency requires the kernel to not be compute-bound, which it is.

This creates a paradox specific to MLA: the same low-rank compression that makes reconstruction weights small enough to be a latency concern *also* makes them small enough to be L2-resident, defeating the primary motivation for weight quantization. Standard LLM linear layers (e.g., FFN projections with weights in the hundreds of MB) do not exhibit this behavior.

(2) INT4 dequantization shifts the bottleneck from memory to compute. The NCU data shows that the INT4 kernel is SM-bound (up to 79% SM utilization) rather than DRAM-bound. Bit masking, shifting, signed extension, and type conversion on every packed byte consume the freed bandwidth headroom. Even when weights exceed L2 and FP16 falls back to HBM, INT4 cannot fully exploit its 4$\times$ data reduction because dequantization saturates the SMs first.

(3) cuBLAS dispatch advantage. `torch.bmm` dispatches via the `nvjet` kernel family with TMA hardware loads (168 registers, L1 bypassed entirely), while our Triton kernel uses standard `ld.global` with 57 registers. Under concurrent L2 contention, INT4 shows greater sensitivity to cache pressure than FP16—consistent with its irregular packed-byte access pattern.

When does INT4 help? The L2 cache barrier is not absolute. In production serving, concurrent operations (FFN GEMMs, attention across multiple layers, multi-tenant request batching)

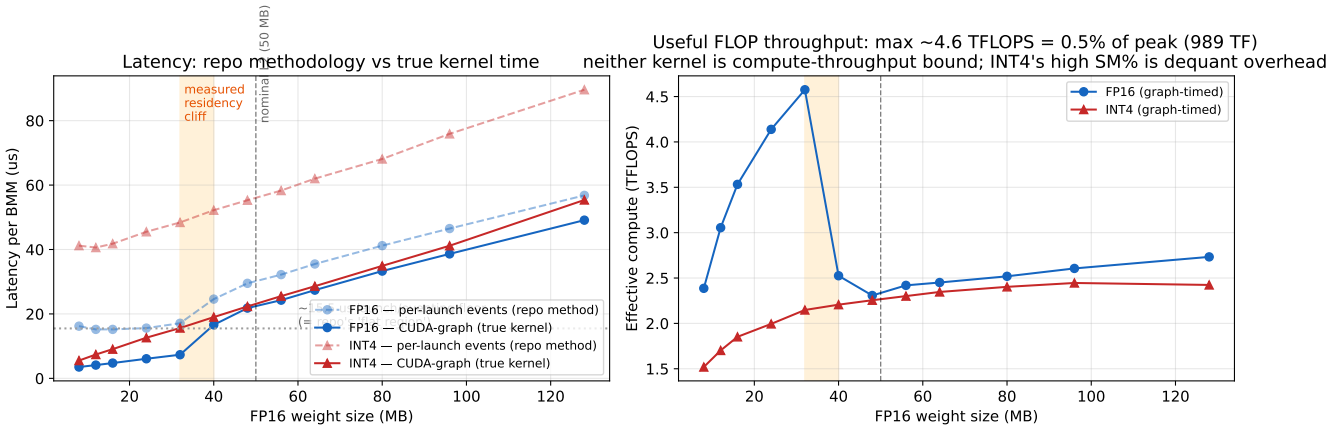

Figure 11: Methodology validation (H100, bs=1, audit stack: torch 2.7/triton 3.3). **Left**: per-launch event timing (repo method, dashed) vs. CUDA-graph timing (true kernel, solid). The ∼15.5 μs launch/eventing floor — not L2 serving — produces the apparent flat region below 32 MB; true kernel latency scales with weight size. **Right**: effective compute throughput peaks at ∼4.6 TFLOPS, 0.5% of the 989 TF peak, so neither kernel is compute-throughput-bound and INT4's high SM utilization is dequantization overhead, not useful FLOPs. Orange band: measured 32–40 MB residency cliff; dashed line: nominal 50 MB L2.

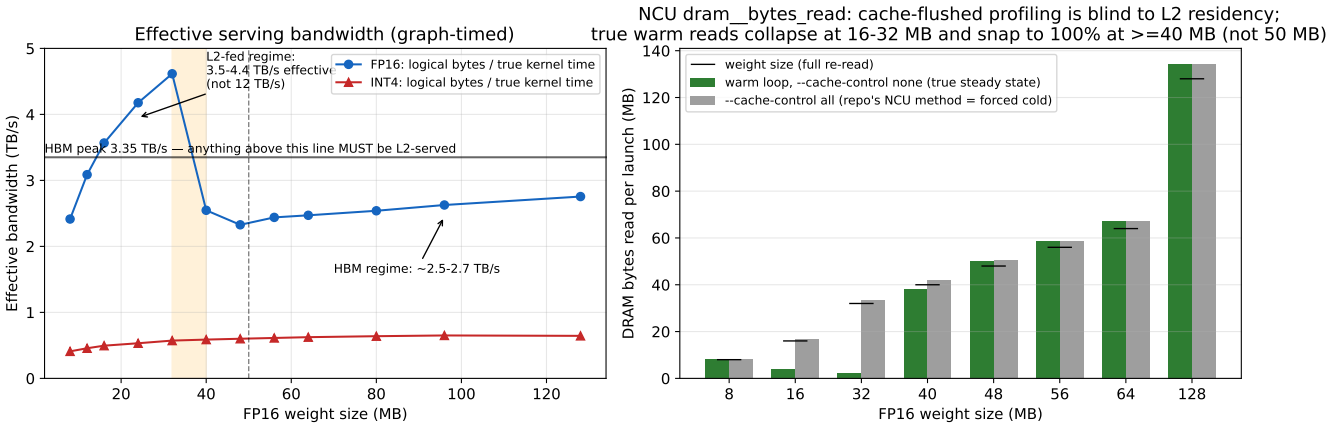

Figure 12: Direct evidence for (partial) L2 serving (H100, bs=1, audit stack). **Left**: effective serving bandwidth (logical bytes / true graph-timed kernel time). Below the residency cliff FP16 exceeds the 3.35 TB/s HBM peak — only possible if L2-served — at an effective 3.5–4.4 TB/s (far below the ∼12 TB/s aggregate L2 figure), and settles to ∼2.5–2.7 TB/s (HBM regime) above it. **Right**: NCU `dram__bytes_read` per launch. Warm-loop counters (`--cache-control none`, green) show DRAM re-reads collapsing far below the full weight size at 16–32 MB and snapping to ∼100% at ≥40 MB; cache-flushed replay (`--cache-control all`, grey) re-reads the full weight at every size and is therefore blind to residency. The cliff is at 32–40 MB (effective capacity ∼36 MB), not the nominal 50 MB.

contend for L2 capacity and may evict reconstruction weights back to HBM. However, our rotation intervention bounds what this can buy for the *current* kernel: with FP16 fully evicted, this W4A16 implementation only reaches parity (1.12× slower at the 16 MB point; 1.0–1.1× across larger sizes). Realizing an actual win under L2 pressure additionally requires a dequantization path that does not saturate the SMs — e.g., INT8 tensor-core MMA with fused dequant — after which fusing reconstruction across layers to exceed L2 capacity, or deploying on GPUs with smaller L2, would let the 4× byte reduction translate into latency.

9.6 A Measured Cache-Aware Roofline

The failures above are failures of the *single-ceiling* roofline. We therefore construct a cache-aware roofline in the spirit of Ilic et al. [9], with two extensions required for microsecond-scale LLM kernels, and with every parameter *measured* on the target GPU (`measure_carm_params.py`) rather than taken from datasheets:

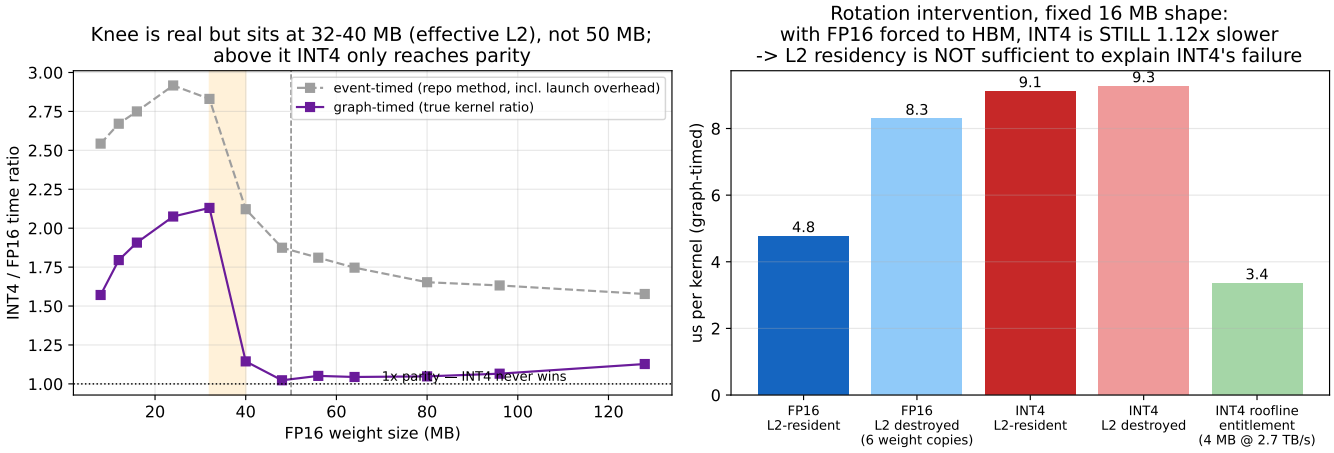

Figure 13: INT4/FP16 ratio and causal decomposition (H100, bs=1, audit stack). **Left**: INT4/FP16 time ratio vs. weight size, event-timed (grey, launch-overhead-inflated) and graph-timed (purple, true kernel). The knee sits at the measured 32–40 MB effective capacity, not the nominal 50 MB; the graph-timed kernel ratio (1.9× near the cliff → 1.0–1.1× at 128 MB) matches the main-stack result, while event-timed magnitudes are stack-dependent (a robustness caveat we flag in the Limitations). **Right**: weight-rotation intervention at fixed 16 MB shape. Forcing FP16 out of L2 (6 weight copies) raises FP16 from 4.8 to 8.3 μs, but INT4 (9.1–9.3 μs, cache-insensitive) is *still* 1.12× slower than evicted FP16 and runs 2.7× above its ∼3.4 μs memory entitlement: L2 residency is necessary but not sufficient to explain INT4's failure; dequantization compute accounts for the rest.

$$t = t_0 + \max\left(\frac{B}{\mathrm{BW(WS)}}, \frac{F}{P_{\text{peak}}}\right),$$
$$\mathrm{BW(WS)} = \begin{cases} \mathrm{BW}^{\text{eff}}_{L2} & \mathrm{WS} < C_{\text{eff}} \\ \mathrm{BW}^{\text{eff}}_{\text{HBM}} & \text{otherwise} \end{cases} \quad (2)$$

Extension 1: capacity-gated bandwidth with effective parameters. The bandwidth ceiling is a function of working-set size, gated at the *measured* effective residency capacity $C_{\text{eff}} \approx 36$ MB (not the nominal 50 MB), with measured tier bandwidths $\mathrm{BW}^{\text{eff}}_{L2} = 5.0$–$6.3$ TB/s (reduction- and GEMM-pattern, respectively; not the ∼12 TB/s aggregate figure) and $\mathrm{BW}^{\text{eff}}_{\text{HBM}} = 3.15$ TB/s.

Extension 2: an explicit per-launch fixed cost t_0 (measured: 2.8 μs graph-captured, 15.4 μs eager). Classic rooflines plot performance against arithmetic intensity alone; for kernels whose data-path time is a few microseconds, t_0 dominates and every small-batch operating point sits far below the loft for reasons no bandwidth ceiling can express. In Figure 14 (panel A) we draw a per-point launch-floor cap F/t_0 that makes this regime explicit.

The INT4 kernel requires a different ceiling: it is bounded by dequantization throughput, not bandwidth. Fitting $t = t'_0 + W_{\text{packed}}\lceil \text{bs}/16 \rceil / R_{\text{dq}}$ to the measured points gives $R_{\text{dq}} \approx 0.50$ TB/s of packed bytes, equivalent to an *in-core ceiling of* ∼30 *TFLOPS — 3% of peak* — which the INT4 operating points pin against at every batch size (Figure 14, panel A). This single number explains the entire INT4 failure: no amount of bandwidth savings can help a kernel whose effective compute ceiling is 3% of the machine.

The combined model predicts FP16 latency across batch sizes 1–512 with 15.5% MAPE and INT4 with 10.9% (Figure 14, panel C; revalidated against re-measured parameters); the largest errors for both kernels are at bs≤16 where the data-path time overlaps the fixed cost, making the model conservative. Both the standard roofline's 3.9× INT4 prediction and the original "L2 at 12 TB/s" explanation are special cases of this model with wrong parameters: the former ignores $\mathrm{BW(WS)}$ and R_{dq}, the latter overestimates both tiers.

9.7 Closing the Loop: W8A8 with INT8 Tensor-Core MMA

The CARM analysis makes a falsifiable prediction: if the dequantization in-core ceiling is removed, quantization should win *exactly* where the working set exceeds C_{eff} (weight bytes served from HBM) and lose where it does not (L2 residency removes the byte-savings upside). We test this constructively with a W8A8 Triton kernel that eliminates per-element dequantization from the inner loop entirely: weights are statically quantized to per-channel symmetric INT8, activations dynamically to per-token INT8 (one auxiliary kernel), the inner loop is a pure `tl.dot(int8, int8)` accumulating in INT32 on Hopper's IMMA tensor cores, and scales are applied once per output element on the final accumulator. Quantizing both operands costs accuracy (relative error ∼0.009 vs. FP16, against $\sim 10^{-3}$ for weight-only), and end-to-end timing includes the activation-quant kernel.

The prediction holds on both sides of the cliff (Table 13, Figure 15): W8A8 is the first quantized kernel in this study to beat cuBLAS FP16 — by 1.4–1.5× at bs=1 for HBM-served weights — and the flip occurs at the measured C_{eff}, not at the nominal 50 MB. At MLA's L2-resident 16 MB operat-

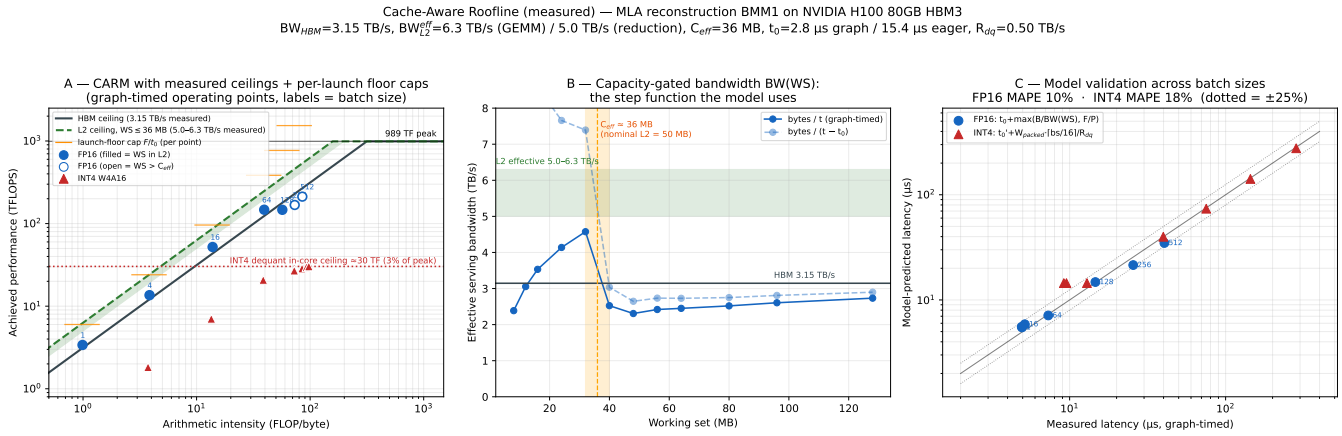

Figure 14: Measured cache-aware roofline for MLA reconstruction (H100, graph-timed). **A**: CARM with measured HBM and L2 ceilings, per-point launch-floor caps, and the fitted INT4 dequant in-core ceiling (∼30 TF); FP16 points (labeled by batch size) are filled when the working set fits C_{eff}. **B**: the capacity-gated bandwidth function $\mathrm{BW}(\mathrm{WS})$ — effective serving bandwidth vs. working set, with the measured residency cliff at 32–40 MB. **C**: model validation, predicted vs. measured latency (dotted $= \pm 25\%$).

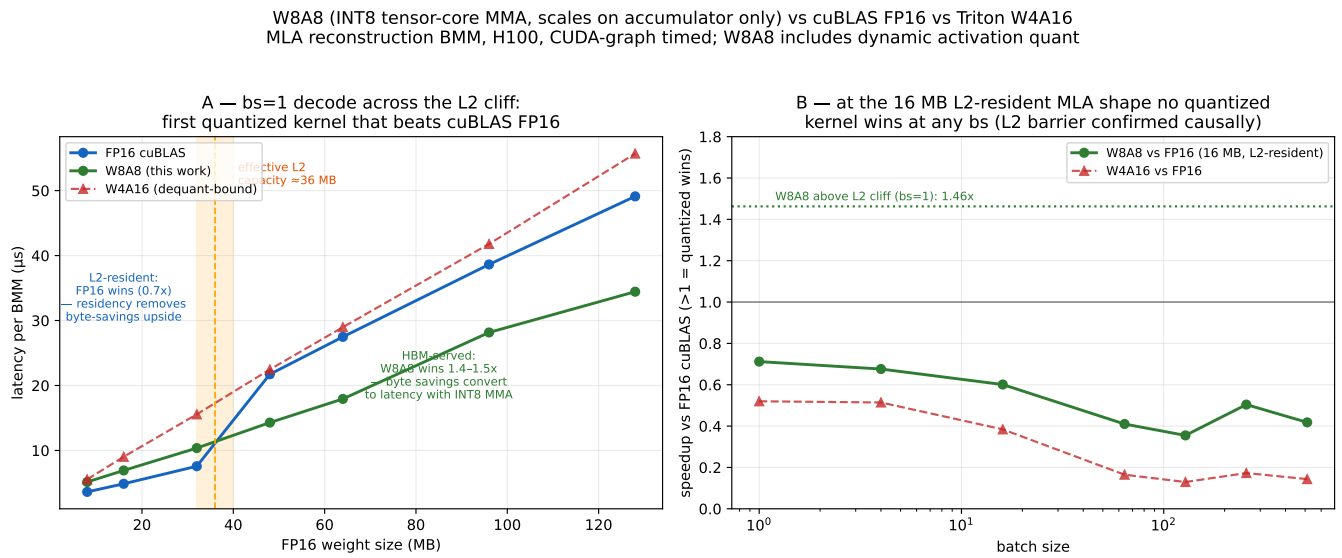

Figure 15: W8A8 INT8-MMA vs. cuBLAS FP16 vs. Triton W4A16 for MLA reconstruction (graph-timed, H100). Left: weight-size sweep at bs=1; right: batch-size sweep at 128 MB. Orange band: measured effective L2 capacity (∼36 MB).

Weights	FP16	W8A8	W4A16	W8A8 speedup
8 MB	3.6 μs	5.1 μs	5.5 μs	0.70×
16 MB (MLA)	4.9	6.9	9.0	0.70×
32 MB	7.6	10.4	15.6	0.73×
48 MB	21.7	14.3	22.5	**1.52**×
64 MB	27.5	17.9	29.0	**1.53**×
96 MB	38.6	28.2	41.8	**1.37**×
128 MB	49.1	34.4	55.7	**1.43**×

Table 13: W8A8 INT8-MMA vs. cuBLAS FP16 vs. Triton W4A16 at bs=1 (graph-timed, H100). The win/loss boundary coincides with the measured effective L2 capacity (∼36 MB).

ing point it still loses (0.70×), as the model requires; and at large batch sizes the workload becomes compute-bound where cuBLAS's TMA pipelines dominate (W8A8 0.35–0.50× at bs≥64). W8A8 beats W4A16 at every measured point. Stacking multiple 16 MB reconstruction layers in one decode step moves the working set across the same cliff (Figure 16): FP16 wins through two layers (32 MB), and W8A8 wins from three layers onward (48 MB, 1.44×), matching CARM's capacity-gated prediction. The corrected deployment rule is therefore three-way and entirely measurable: *FP16 when the working set is L2-served; W8A8 INT8-MMA when weight-byte-bound from HBM; FP16 (absent tuned INT8 pipelines) when compute-bound.*

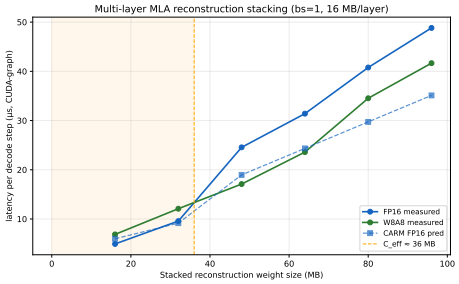

Figure 16: Multi-layer MLA reconstruction stacking (bs=1, 16 MB/layer, graph-timed). Stacking 1–6 layers sweeps the per-step working set from 16 to 96 MB; the FP16→W8A8 crossover occurs between two and three layers (32→48 MB), bracketing the measured $C_{\text{eff}} \approx 36$ MB. Dashed: CARM FP16 prediction.

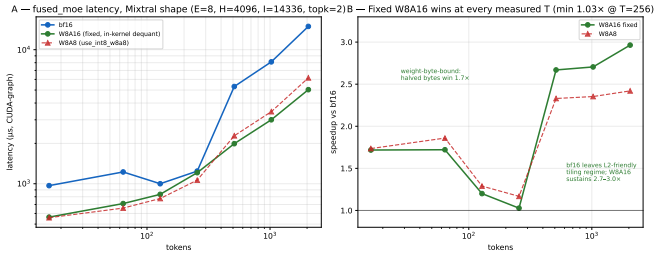

Figure 17: FlagGems `fused_moe`, Mixtral shape, graph-timed (T=16–2048). **A**: latency for bf16, fixed W8A16 (in-kernel dequant), and W8A8. **B**: speedup over bf16 — the fixed W8A16 kernel wins at every measured token count, 2.7–3.0× at T≥512.

9.8 Generalization: Mixed-Precision MoE (FlagGems)

The same regime structure appears in an independent workload class. We analyzed FlagGems' mixed-precision `fused_moe` (PR #2336) on Mixtral-shape MoE GEMMs (E=8, H=4096, I=14336, top-k=2). The shipped W8A16 path host-dequantized *all* expert weights on every call (∼10 GB of traffic, a flat 8.5 ms at every token count); a one-conditional fix routing per-channel INT8 through the kernel's existing in-loop conversion yields 1.7× over bf16 at 16–64 tokens and **2.7–3.0× at T≥512**, with no measured crossover back to bf16 through T=2048 (minimum 1.03× at T=256; Figure 17). Warm-state counters at T=512 show the fixed W8A16 kernel at 24% DRAM / 32% SM utilization — stalled on the int8→fp16 conversion feeding `tl.dot` — an in-core conversion ceiling of ∼305 TF, structurally identical to the MLA W4A16 ceiling (∼30 TF) but 10× higher because conversion is vectorized and tensor cores perform the math. The regime boundary remains predictable from CARM's parameters: weight-only quantization wins while the workload is weight-byte-bound; at this MoE shape the per-expert working set (∼1.4 GB total) never becomes L2-served, so the fixed kernel never re-enters the losing regime.

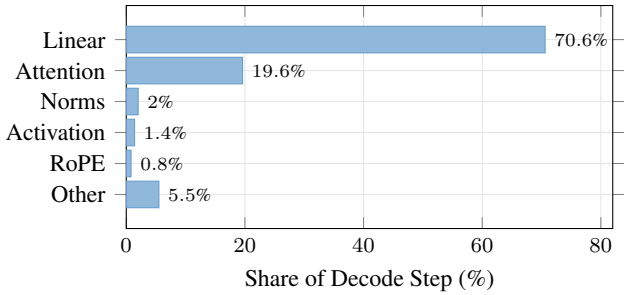

Figure 18: Decode step decomposition (Qwen2.5-7B, bs=64). Linear layers dominate at 71%; attention is ∼20%. For MLA, reconstruction BMMs add to this: at bs=1, reconstruction (2.17 ms) exceeds attention (1.40 ms).

10 End-to-End Context of the Case Study

To quantify the system-level impact of reconstruction, we contrast MLA against a standard GQA decode workload.

Table 14: Decode step decomposition, Qwen2.5-7B, bs=64.

Component	Time (μs)	Share
Linear layers (GEMMs)	**5,079**	**70.6%**
Attention (FlashInfer)	1,413	19.6%
Norms (RMSNorm)	145	2.0%
Activation (SiLU)	98	1.4%
RoPE	57	0.8%
Other	399	5.5%

Attention constitutes ∼20% of decode time for GQA models (Table 14, Figure 18). For MLA models like DeepSeek-V3, reconstruction overhead adds a further component: at bs=1, reconstruction GEMMs (2.17 ms across 61 layers) exceed the attention kernel cost (1.40 ms), making reconstruction the dominant attention-layer bottleneck. Targeted optimization of reconstruction, whether through INT4 quantization or kernel fusion, is therefore higher-impact than further attention kernel tuning.

11 Reproducibility

Table 15: Reproducibility across 3 independent trials (warmup=20, iters=200).

Config	T1	T2	T3	CV
FI dec bs=1	0.029	0.030	0.030	3.4%
Tri dec bs=1	0.056	0.059	0.057	2.6%
FI dec bs=64	0.187	0.187	0.187	**<0.1%**
Tri dec bs=64	0.216	0.216	0.216	**<0.1%**
FI dec bs=256	0.719	0.718	0.718	**<0.1%**
Tri dec bs=256	0.813	0.813	0.813	**<0.1%**

At bs≥64, results are deterministic (<0.1% CV) (Table 15). At bs=1, ∼3% variance arises from GPU boost clock fluctuations on short (∼28 μs) kernels. Relative speedup ratios are stable across all trials.

12 Discussion

Our analysis reveals a chain of findings: MLA's KV compression shifts the attention-layer bottleneck to reconstruction GEMMs (61% at bs=1); these are memory-bound at all practical batch sizes; INT4 quantization preserves quality but fails to deliver speedup due to L2 cache residency. We now discuss the implications.

12.1 Implications for Compiler-Generated Attention

The prefill gap is fundamental to the compiler architecture: Triton's MLIR→PTX pipeline generates `ld.global` instructions because it has no TMA code generation path for attention patterns. Triton 3.x introduced `tl.experimental_descriptor_load` for TMA access, but this requires manual descriptor setup that attention kernels have not adopted. Closing the prefill gap requires either:

1. Triton compiler support for automatic TMA lowering of attention-shaped loads (research problem).

2. Manual TMA integration in the Triton kernel source (engineering, loses portability).

3. A higher-level DSL that targets both TMA and global load paths depending on the GPU architecture.

The decode gap (1.1×) is more tractable: Triton's two-phase split could be fused, and register tuning (`num_warps`, tiling factors) could recover some bandwidth.

12.2 The Occupancy Paradox as Design Principle

FlashInfer's decode kernel shows that for memory-bound workloads, *maximizing bandwidth per warp* outperforms *maximizing warp count*. At 183 registers, FlashInfer has 2.4× fewer active warps than Triton, yet extracts 10% more HBM bandwidth. This is consistent with the principle that memory-bound kernels benefit more from coalesced access patterns and prefetching than from raw occupancy [13].

12.3 MLA: Compression Shifts the Bottleneck

MLA's 7.1× KV traffic reduction makes the attention kernel faster but creates a new bottleneck in the reconstruction GEMMs that were previously negligible. At bs=1, reconstruction costs 61% of attention-layer time. This illustrates a general systems principle: optimizations that reduce data movement can shift workloads into regimes where cache hierarchy, rather than raw bandwidth, determines performance. The implication for MLA specifically is that reconstruction overhead must be co-optimized with attention, not ignored.

12.4 Selective Quantization: Quality Result

Despite the kernel limitations, reconstruction weights tolerate quantization well: +0.051 PPL for selective INT4 vs. +6.057 for naive INT4 of all weights. For future MLA serving systems, **reconstruction weights are the natural first target for aggressive quantization**, pending kernel support.

12.5 Limitations

- The quantized Triton kernels drift off their own rooflines at $T \geq 64$ (up to 81% MAPE) while cuBLAS stays at $\sim$17%; the model prices the physics, not Triton codegen quality at large M. All headline claims are confined to the decode regime or reproduced on tuned CUDA.

- The A100 constants are now harness-measured and the primary transfer target graph-timed on the same GPU, retiring the estimated-constants caveat; what remains is that the measured card is the 40GB (HBM2) variant while the 2026-06 case-study data is from the 80GB (HBM2e), and that the below-gate fp16 residual (44% MAPE) indicates a model-form weakness for small L2-resident kernels that measured constants do not fix.

- The dispatch cost model's KV-per-token figure assumes the Qwen3.6-27B hybrid-attention layout (16 full-attention layers); dense-attention models shift the concurrency arithmetic, not the policy ordering.

- Serving-contention measurements use concurrent GEMM streams as a proxy for full-engine co-residency; the staged served A/B (Section 11) closes this gap.

- Single GPU; multi-GPU tensor-parallel workloads may show different ratios due to all-reduce overlap.

- Sections 8 onward predate clock locking; absolute numbers there carry $\sim$3% uncertainty at small batch sizes. The gate sweep (Section 3) and all 2026-08 measurements are clock-locked at 1755 MHz.

- Per-launch CUDA-event timing carries a $\sim$15.5 μs floor; eager-mode bs$\leq$16 latencies (and the 61% reconstruction share) are launch-overhead-inflated. Serving engines that capture decode in CUDA graphs will see smaller absolute reconstruction overhead (graph-timed: $\sim$4.8 μs per BMM vs. $\sim$17 μs eager), though the recon-vs-attention ratio is less affected since both sides carry the floor.

- Replay-based NCU counters are cold-cache (`--cache-control all`); only the `--cache-control none` warm-loop counters and timing interventions speak to residency.

- INT4/FP16 ratios are software-stack-sensitive: PyTorch 2.9.1/Triton 3.5.1 yields 1.86× (16 MB) → 1.08× (128 MB) event-timed, while PyTorch 2.7/Triton 3.3 yields 2.75× → 1.58× on identical hardware. Kernel-level (graph-timed)

ratios are more stable (1.9× → 1.0–1.1×). Directional conclusions transfer; magnitudes do not.

- cuBLAS dispatches different `nvjet` tile variants across the weight-size sweep; the FP16 curve mixes kernel implementations.
- NCU local memory counters unavailable on Hopper (`n/a`); spill analysis relies on PTX/CUBIN inspection.
- INT4 kernel benchmarked in Triton only; a CUDA-native implementation with INT8 tensor cores may achieve different results.
- Perplexity evaluated on DeepSeek-V2-Lite (15.7B); larger models may show different quantization sensitivity.
- One model architecture per attention type.

13 Related Work

Positioning (added 2026-08, DIRECTION.md P0). Four lines of work sit nearest to ours. *tritonBLAS* (arXiv:2512.04226) uses an analytical model of the cache hierarchy plus shape to select GEMM tile/blocking configurations in Triton at zero tuning cost; it selects *configuration at fixed precision*, whereas we select *precision*, and our capacity gate is a natural extension of their model family. *QServe* (arXiv:2405.04532) identified main-loop dequantization overhead as a kernel-design obstacle and engineered W4A8 around it; we instead measure it as a composable, predictive roofline ceiling (r_{dequant}, the in-core term of Section 9.6) used at dispatch time. *MARLIN* (arXiv:2408.11743) documents the near-ideal $\sim 4\times$ weight-only speedup at batch ≤ 32 and its collapse beyond batch ~ 64; our model explains *why* (capacity gate below, compute roofline above) and predicts *where* the crossover lands per shape and architecture — MARLIN's headline number is the far-field limit of the same model whose near field is our L2-resident null result. W4A4 kernel designs such as APEX4 (arXiv:2606.08761) are orthogonal: they build the kernels among which we dispatch. Microbenchmark-parameterized cross-architecture performance models (arXiv:2605.04178) share our portability mechanism but lack both the capacity term and the precision decision; LLM roofline studies (arXiv:2402.16363, 2602.11506) analyze precision without a capacity term, which is precisely the gap this paper fills.

Attention kernels. FlashAttention [3] and FlashAttention-2 [2] introduced tiling-based exact attention with $O(N)$ memory. FlashInfer [15] extends this with Hopper-native TMA support. Triton [12] enables Python-level GPU kernel development. SGLang [16] integrates both backends for LLM serving.

MLA architectures. DeepSeek-V2 [4] introduced Multi-head Latent Attention; DeepSeek-V3 [5] scaled it to 671B parameters with 128 attention heads. While both papers describe the weight-absorbed reconstruction mechanism, neither quantifies its runtime cost relative to the attention kernel.

Performance modeling. The roofline model [14] is the standard framework for reasoning about compute- vs. memory-bound kernels. Ilic et al. [9] extended it to cache-aware form with per-level bandwidth ceilings. Our L2 cache barrier finding (Section 9.5) exposes a failure mode of single-level roofline analysis — when working sets (partially) fit in L2, the effective bandwidth ceiling shifts from HBM toward L2 — and Section 9.6 contributes a measured CARM variant with a capacity-gated ceiling, an explicit per-launch fixed cost, and a fitted dequantization in-core ceiling, validated to 11–16% MAPE on the reconstruction kernels. Volkov [13] showed that occupancy is not always the primary performance factor, motivating our analysis of FlashInfer's register-heavy approach.

Weight quantization. GPTQ [6] and AWQ [10] enable post-training INT4 weight quantization for LLM inference. KVQuant [8] applies quantization to the KV cache itself, targeting the memory footprint that MLA addresses architecturally. QuaRot [1] uses rotation to eliminate outliers before 4-bit quantization. All prior quantization work targets standard linear layers (FFN, attention projections), not the reconstruction matrices specific to MLA. To our knowledge, no prior public work quantifies the reconstruction GEMM overhead in MLA or evaluates selective INT4 quantization of these weights.

14 Conclusion

Whether quantization pays for a given GEMM is decided by a capacity condition standard roofline analysis cannot express: once the weight working set is last-level-cache resident, the traffic savings that motivate quantization vanish, and any in-core dequantization cost flips the quantized path to a net loss. We measured the gate directly — the win/loss sign flips between 32 and 40 MB, bracketing the measured 36 MB effective capacity of H100's nominal 50 MB L2 — and showed that the literature's apparently contradictory results (MARLIN's $\sim 4\times$ at small batch; our L2-resident null results) are the far and near fields of one three-parameter model.

The model earns its parameters. It predicts the tuned baseline to 12–21% everywhere and quantized kernels to 8–39% in the decode regime where dispatch operates; its dequant-ceiling term correctly caps W4A16 at parity in the far field; its parameters are measurable on any CUDA-semantics backend by a portable harness, transfer to A100 at 13–19% MAPE above the gate (the dispatch-relevant regime) with only a two-point per-kernel calibration, and provably cannot be extrapolated by hardware scaling ratios. Its boundary conditions are stated rather than hidden: serving contention collapses the L2-resident regime as a step function, so isolated-microbenchmark crossovers are optimistic in production, and five measured kernel mechanisms mean dispatch must compose the model with per-kernel predicates.

The dispatch cost analysis converts the model into a shippable policy: dual-format residency and on-line repacking are respectively memory-infeasible and switch-period-limited, so weights should be stored quantized-primary, and the runtime decision re-

duces to *pay the dequant ceiling or take the quantized-compute path*, evaluated per shape against measured C_{eff}. As LLC capacities grow (40→50→126→192 MB across recent generations), the regime where quantization stops paying grows with them; the model, unlike any fixed heuristic, moves with the measurement.

All profiling scripts, raw data (JSON alongside every figure), and evaluation code are publicly available for reproduction.

References

[1] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. Quarot: Outlier-free 4-bit inference in rotated llms. *arXiv preprint arXiv:2404.00456*, 2024.

[2] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In *ICLR*, 2024.

[3] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *NeurIPS*, 2022.

[4] DeepSeek-AI. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[5] DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[6] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. In *ICLR*, 2023.

[7] Elias Frantar, Roberto L. Castro, Jiale Chen, Torsten Hoefler, and Dan Alistarh. Marlin: Mixed-precision auto-regressive parallel inference on large language models. *arXiv preprint arXiv:2408.11743*, 2024.

[8] Coleman Hooper, Sehoon Kim, Hiva Mohammadi, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

[9] Aleksandar Ilic, Frederico Pratas, and Leonel Sousa. Cache-aware roofline model: Upgrading the loft. *IEEE Computer Architecture Letters*, 13(1):21–24, 2014.

[10] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for llm compression and acceleration. In *MLSys*, 2024.

[11] NVIDIA. Nvidia h100 tensor core gpu architecture. `https://resources.nvidia.com/en-us-tensor-core`, 2022.

[12] Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *MAPL*, 2019.

[13] Vasily Volkov. Better performance at lower occupancy. In *GPU Technology Conference*, 2010.

[14] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

[15] Zihao Ye, Lequn Lai, Ruihang Shao, Wuwei Zheng, and Tianqi Chen. Flashinfer: Efficient and customizable attention engine for llm inference serving. In *MLSys*, 2025.

[16] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kober, Yineng Shi, et al. Sglang: Efficient execution of structured language model programs. In *NeurIPS*, 2024.